Instituto Tecnológico de las Américas (ITLA)

Tecnólogo en Desarrollo de Software

Capa de Aplicación (Business Logic Layer)

Sistema de Gestión de Gimnasio (SGG)

Servicios de aplicación, dependencias en IOC y arquitectura resultante

Estudiante: Misael Smith Dirocie Peña

Matrícula: 2025-1007

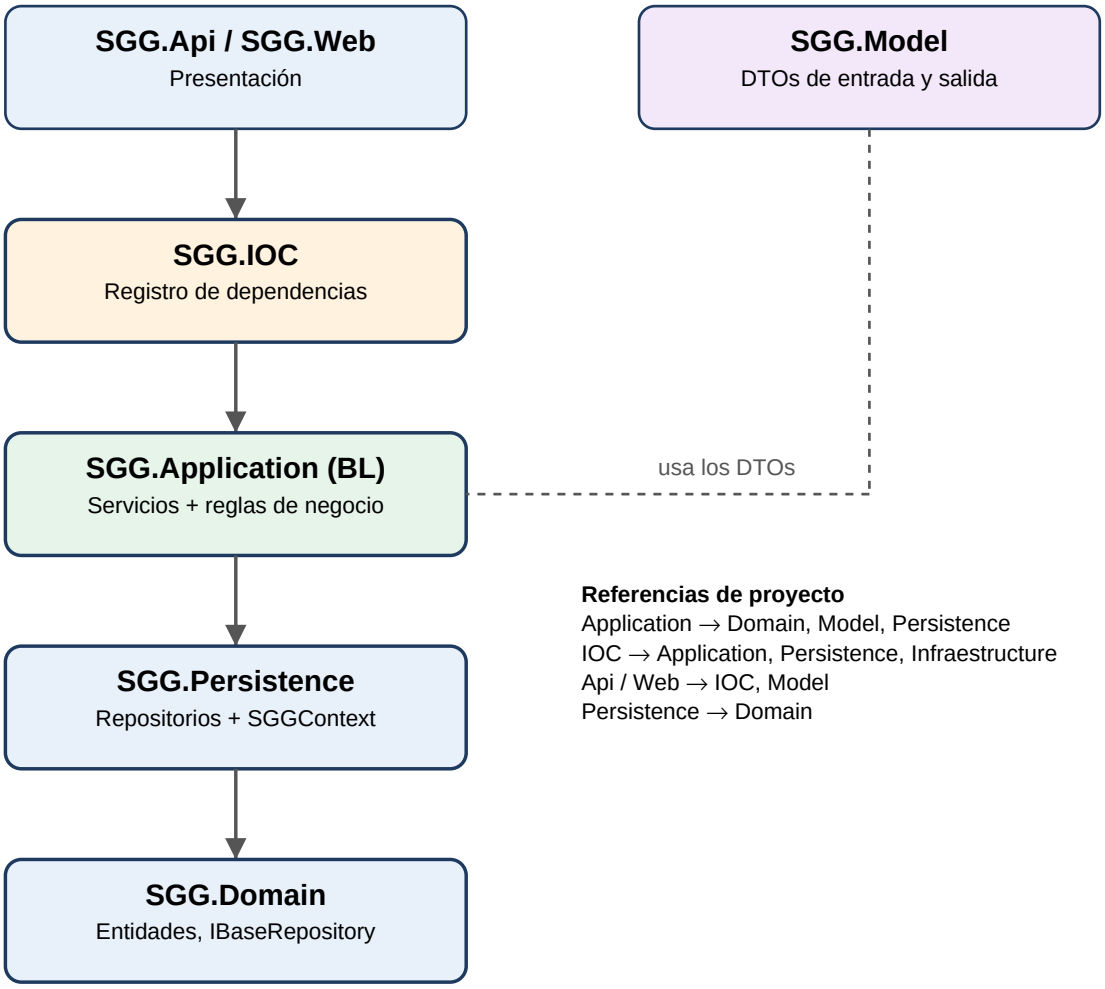
Asignatura: Programación II (2026-C-3)

Profesor: Francis Ramírez

Fecha: 2 de noviembre de 2026

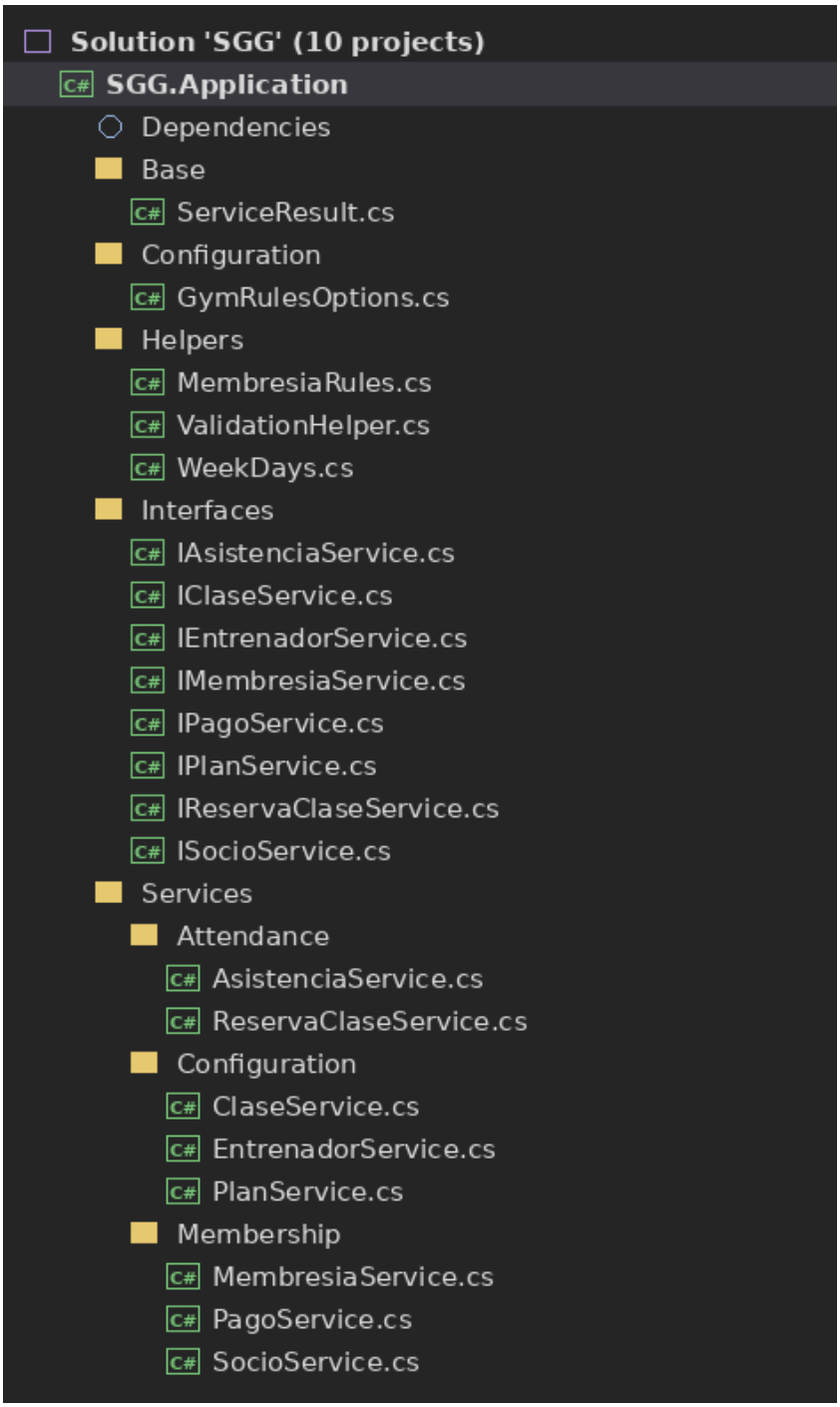
1. Arquitectura resultante

La capa de aplicación (**SGG.Application**) implementa la funcionalidad principal del sistema y encapsula la lógica empresarial: validaciones, reglas de negocio y coordinación de los repositorios. Las capas de presentación (Api y Web) solo consumen interfaces de servicio; la creación de objetos queda en **SGG.IOC**.



Flujo de una operación: el cliente envía un DTO (SGG.Model) → el servicio de aplicación valida y aplica las reglas → usa los repositorios (`IPlanRepository`, etc.) → devuelve un `OperationResult` con mensaje y datos. Las eliminaciones son lógicas (`Status = false`) para conservar el historial.

2. Estructura de SGG.Application



Base
ServiceResult: fábrica de OperationResult y manejo uniforme de errores con log.

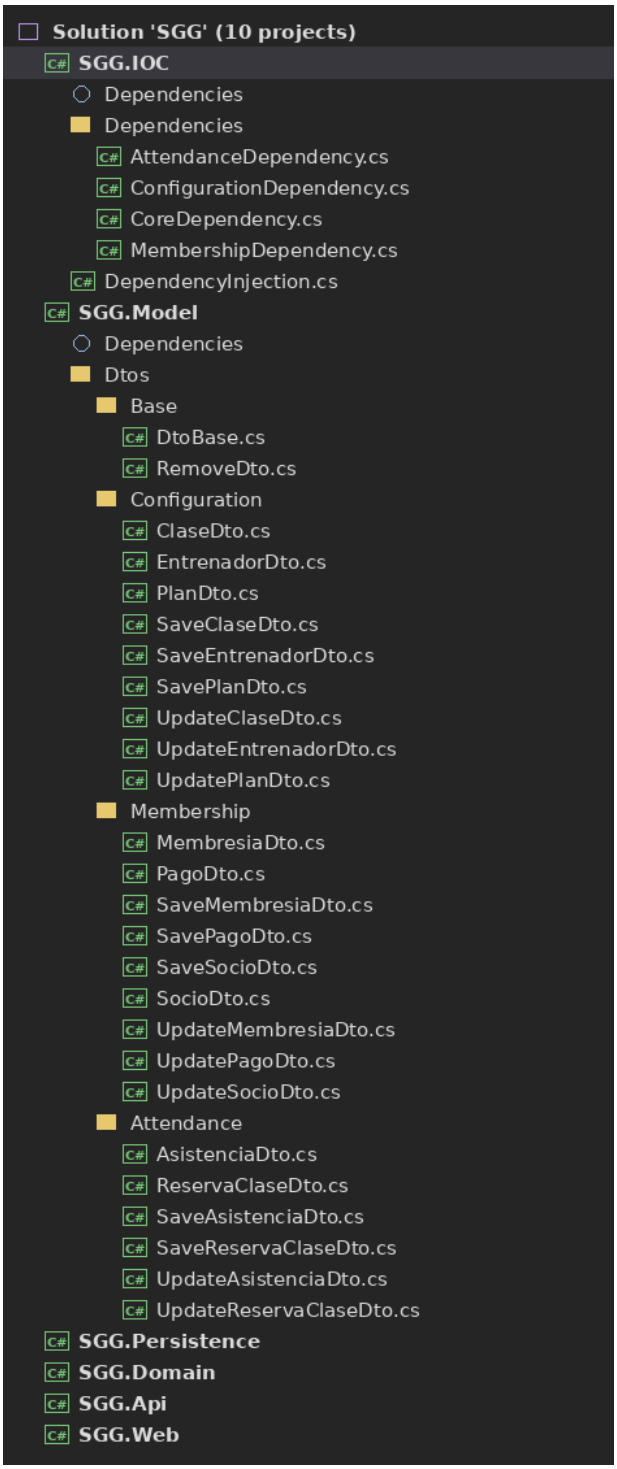
Configuration
GymRulesOptions: reglas configurables (métodos de pago, capacidad máxima de clase, reservas activas por socio) leídas de la sección GymRules.

Helpers
Validaciones reutilizables (cédula, correo, teléfono), días de la semana y regla de membresía vigente.

Interfaces
Un contrato por servicio (GetAll, GetById, Save, Update, Remove y consultas por relación).

Services
Una carpeta por módulo (schema), igual que en la capa de persistencia: Configuration, Membership y Attendance.

3. SGG.IOC y SGG.Model



SGG.IOC
Una clase de dependencias por módulo (repositorios + servicios) y una de núcleo (DbContext y opciones). DependencyInjection.AddSGGDependencies las reúne y se llama desde Program.cs de la Api.

SGG.Model
Por cada entidad: un DTO de lectura (XDto), uno de creación (SaveXDto) y uno de edición (UpdateXDto, hereda de Save y agrega Id). DtoBase aporta UserId y RemoveDto sirve para todas las eliminaciones.

4. Servicios por módulo y reglas de negocio

Módulo	Servicio	Reglas de negocio principales
Configuration	PlanService	Nombre único; precio y duración mayores que cero; no se elimina un plan con membresías vigentes.
	EntrenadorService	Nombre y apellido requeridos; teléfono válido; no se elimina si tiene clases asignadas.
	ClaseService	Entrenador existente; capacidad dentro del máximo; día válido; hora inicio menor que fin; sin choque de horario del entrenador; no se elimina con reservas pendientes.
Membership	SocioService	Cédula (11 dígitos) y correo válidos y únicos; fecha de nacimiento coherente; no se elimina con membresía vigente.
	MembresiaService	Socio y plan existentes; la fecha de vencimiento se calcula con la duración del plan; sin membresías solapadas; no se elimina con pagos.
	PagoService	Membresía existente; monto mayor que cero; fecha no futura; método de pago dentro de los permitidos.
Attendance	AsistenciaService	Socio con membresía vigente en la fecha; clase existente (si se indica); sin asistencia duplicada el mismo día.
	ReservaClaseService	La fecha coincide con el día de la clase; membresía vigente; sin reserva duplicada; respeta la capacidad y el máximo de reservas activas; no se cancela una clase ya iniciada.

Todos los servicios devuelven `OperationResult` y registran los errores inesperados con `ILogger`.

5. Dependencias en IOC y entrega

```
// SGG.IOC/Dependencies/MembershipDependency.cs
services.AddScoped<ISocioRepository, SocioRepository>();
services.AddScoped<IMembresiaRepository, MembresiaRepository>();
services.AddScoped<IPagoRepository, PagoRepository>();

services.AddScoped<ISocioService, SocioService>();
services.AddScoped<IMembresiaService, MembresiaService>();
services.AddScoped<IPagoService, PagoService>();

// SGG.Api/Program.cs
builder.Services.AddSGGDependencies(builder.Configuration);
```

Cada módulo registra sus repositorios y servicios en su propio archivo (ConfigurationDependency, MembershipDependency, AttendanceDependency), de modo que cada integrante trabaja en su rama sin chocar con los demás. CoreDependency registra SGGContext (cadena de conexión `SGGContext`) y las reglas GymRules.

Repositorio: https://github.com/Misael2025-1007/SGG

Ramas: feature/configuration, feature/membership, feature/attendance y develop (con todo fusionado).